



# 汇编语言

---

16 总复习

张青

ieqzhang@zzu.edu.cn

# 第1章 汇编语言与计算机系统

- 掌握汇编语言的特点及应用场景
- 能说明计算机系统组成及各部分的作用
- 清楚计算机系统的层次结构，尤其是指令集架构（ISA）的含义和作用
- 能说出程序的开发步骤及使用的软件
- 掌握编译器、汇编器、连接器的作用
- 能正确使用GCC命令实现C语言转换为汇编语言、目标文件和可执行文件
- 能说出可执行程序的执行过程
- 掌握汇编语言的语句格式及各个部分的要求
- 能写出GAS汇编语言程序框架

# 汇编语言和高级语言的比较

- 汇编语言与处理器密切相关，可移植性较差
  - 高级语言与具体计算机无关，便于移植
- 汇编语言功能有限、涉及硬件细节，编写繁琐困难
  - 高级语言功能强大，语法自然，容易掌握和应用
- 汇编语言的优势：直接、有效地控制硬件，容易产生运行速度快、指令序列短小的高效率目标程序
  - 高级语言不易直接控制计算机的各种操作，编译程序产生的目标程序往往比较庞大

混合编程、取长补短

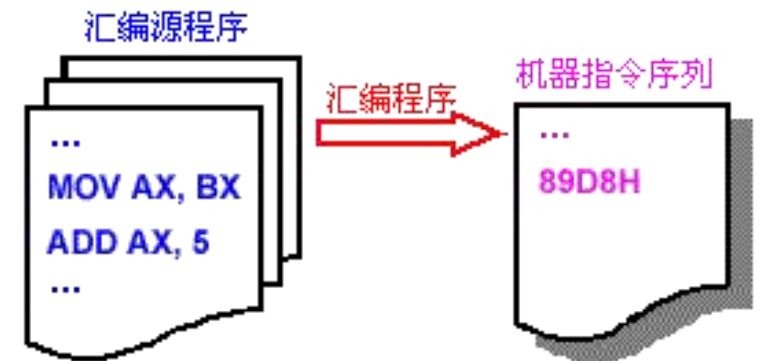
学习汇编语言的意义



# 汇编语言的优势



- 应用价值
  - 编写运行速度快、代码量小的高效程序
  - 编写低层、直接控制硬件的程序
  - 加解密、病毒防治等信息安全程序的编程分析
  - 更好地使用高级语言编程
  - 国产信息化及信息安全的基础
    - 没有合适的高级语言，或只能采用汇编语言时
    - 鲲鹏处理器、神威、龙芯移植



# 计算机系统的层次结构



# 指令集体系架构ISA



- 指令集体系架构（Instruction Set Architecture, ISA）也叫做计算机体系结构。
- IAS位于硬件和软件的交界面上，是整个计算机系统的核心部分，是构建计算机硬件和软件的规范。
  - 硬件所具有的概念性的结构和功能特性都由ISA集中体现
  - 软件要通过ISA在硬件上执行
- 目前典型的指令集架构：
  - CISC（Complex Instruction Set Computer, 复杂指令集计算机）
  - RISC（Reduced Instruction Set Computer, 精简指令集计算机）

# 可执行文件的开发过程

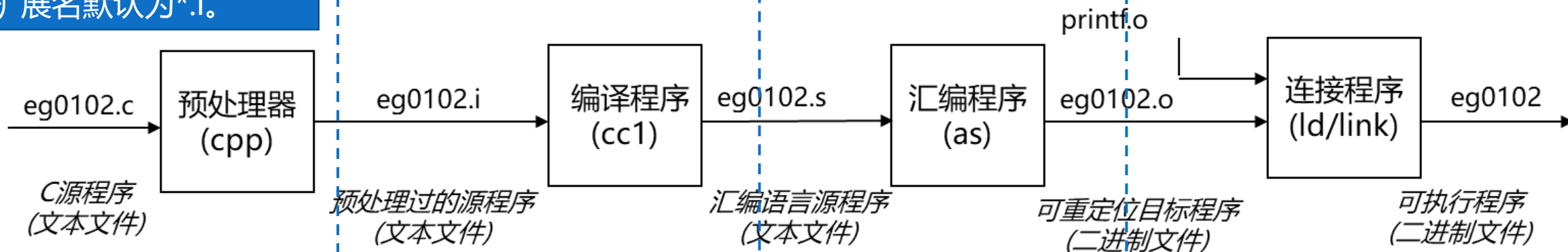
- 使用下列命令可以生成可执行文件：  
`gcc -Og -o eg0102 eg0102.c`

`gcc -E -o eg0102.i eg0102.c`

**预处理阶段**对源程序中的预处理语言进行处理，输出还是源程序文件，扩展名默认为\*.i。

`gcc -c -o eg0102s.o eg0102.s`

**汇编阶段**对汇编语言源程序进行汇编生成定位目标程序（二进制文件），扩展名默认为\*.o



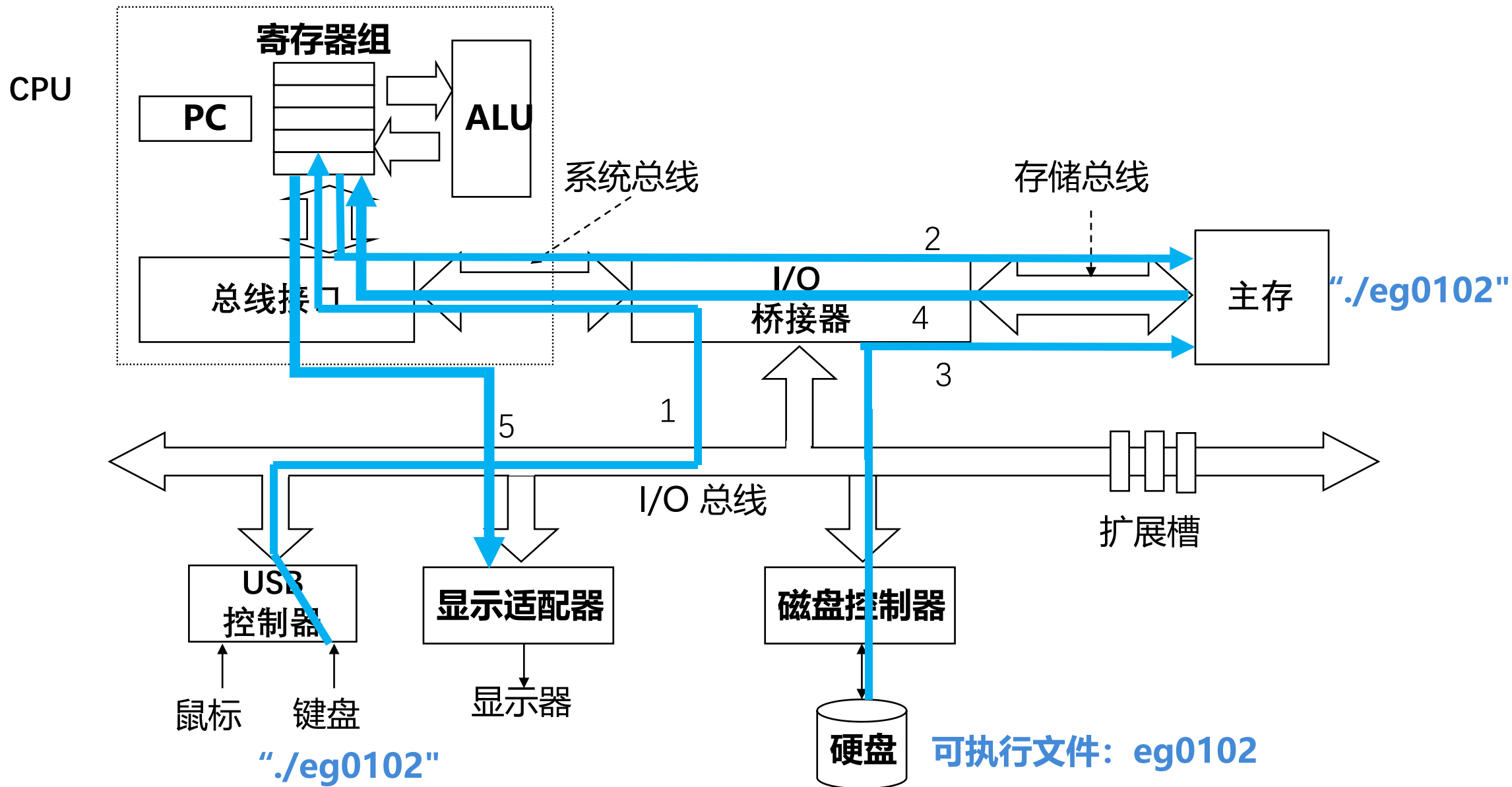
**编译阶段**对预处理后的源程序进行编译，生成汇编语言源程序文件，扩展名默认为\*.s。

**连接阶段**将多个可重定位目标程序和标准函数库合并为可执行文件

`gcc -S -Og -masm=intel -o eg0102.s eg0102.c`

`gcc -o eg0102 eg0102.o`

# 程序执行过程





# GAS语句格式



**标号: 助记符 操作数 #注释**

表示处理器指令在主存中的逻辑地址的标识符

表示功能的记忆符号  
真指令：处理器命令  
伪指令：汇编器命令

参数，初值表

- 自定义标识符：
  - 以字母或下划线开始，不能以数字开头，大小写敏感
  - 由字母、数字及规定的特殊符号（“\_”、“.”、“\$”）组成
  - 自定义标识符必须是唯一的，不能用汇编程序的保留字
  - 应尽量具有描述性并易于理解，不建议使用特殊符号开头

# 程序框架



```
/* AS汇编语言基本框架 */
```

```
.intel_syntax noprefix # 采用Intel语法, 面向X86处理器时使用
```

```
.data # 定义数据段
```

```
# 数据定义 (数据待填)
```

```
.text # 定义代码段
```

```
.globl main
```

```
main: # 程序执行起始位置
```

```
#主程序 (指令待填)
```

```
ret # 程序正常执行结束
```

```
#子程序 (指令待填)
```

## 第2章 数据表示与存储

- 能进行不同数制及编码之间的转换
- 掌握C语言数据类型与汇编语言变量之间的关系
- 掌握GAS伪指令：定义变量、常量、分配空间、地址对齐
- 清楚多字节数据存储的大端方式与小端方式
- 会分析数据在内存中的存储及内存中存储内容对应的数据
- 会用GAS生成列表文件并分析列表文件的内容

# eg0214.c对应的汇编语言程序



提问：如何将eg0214.c 转换为汇编语言程序eg0214.S?

```
int x = -1;
unsigned int u = 2147483648;
long l = -1;
unsigned long ul = -1;
short s = -1;
unsigned short us = -1;
char c = 'A';
unsigned char uc = 200;
char str[] = "Hello Assembly!";
char str1[] = {'C','h','i','n','a','\0'};
char *ps = str;
```

|       |        |                    |
|-------|--------|--------------------|
| x:    | .long  | -1                 |
| u:    | .long  | -2147483648        |
| l:    | .long  | -1                 |
| ul:   | .long  | -1                 |
| s:    | .word  | -1                 |
| us:   | .word  | -1                 |
| c:    | .byte  | 65                 |
| uc:   | .byte  | -56                |
| str1: | .ascii | "China\0"          |
| str:  | .asciz | "Hello Assembly! " |
| ps:   | .quad  | str                |

c语言中long类型，在Windows（32位和64位）下是4字节，在64位Linux下是8字节

下面程序的输出结果是什么？说明理由。

```
#include <stdio.h>
int x = -2;
unsigned int y= 0;
int main (){
    printf("%x,%d",x+1, y-1);
    return 0;
}
```

输出结果：

ffffffff,-1

# 常量符号定义

- 常量也可以用符号表示，称为符号常量。

- .equ

- .equ 定义的符号常量不可修改

.equ 符号名, 数值表达式/<字符串>

```
.equ NULL, 0  
message: .ascii "hello, world"  
.equ msglen, .-message
```

- .set

- 可重新赋值

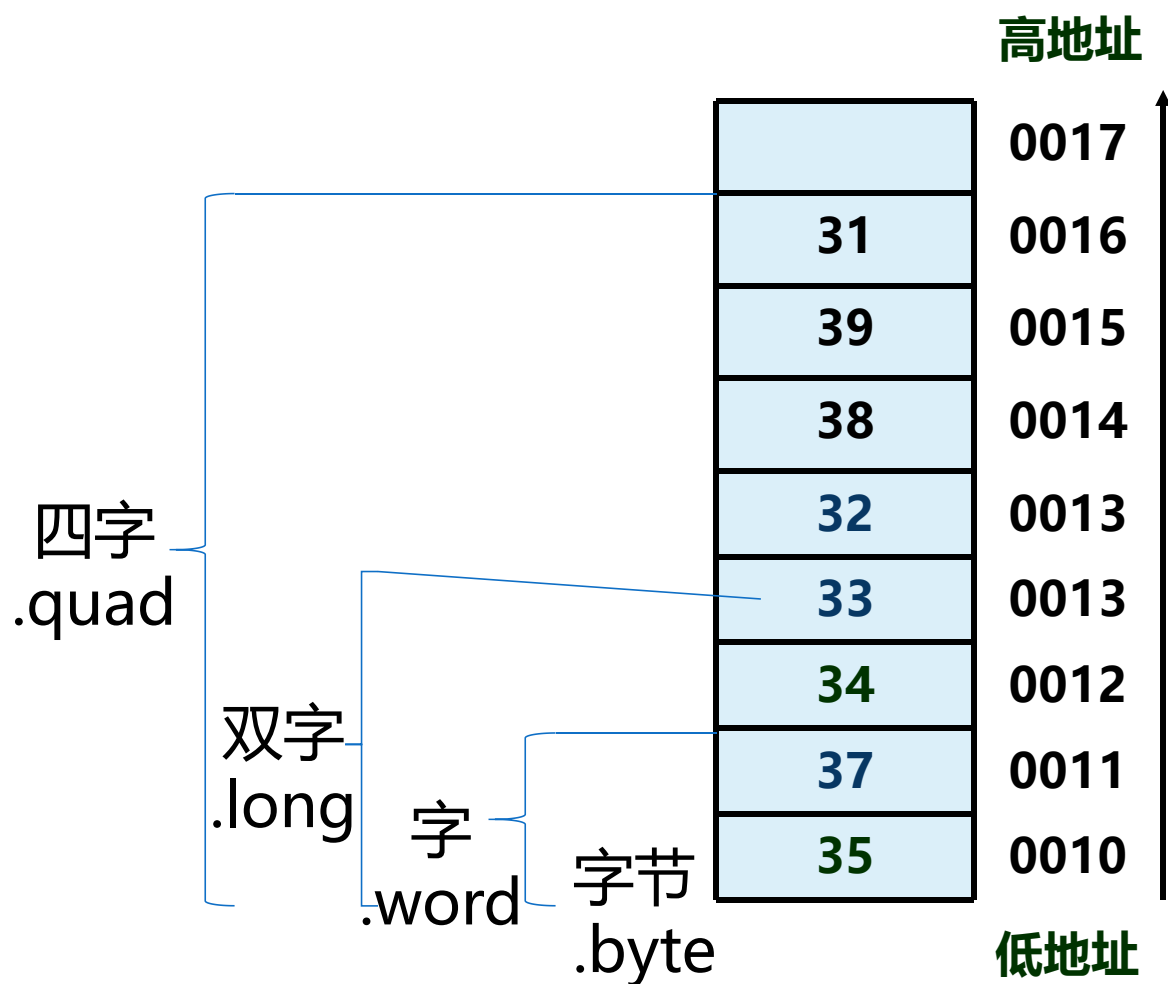
```
.set char, 'A'  
.set char, char+1
```

# 变量定义伪指令

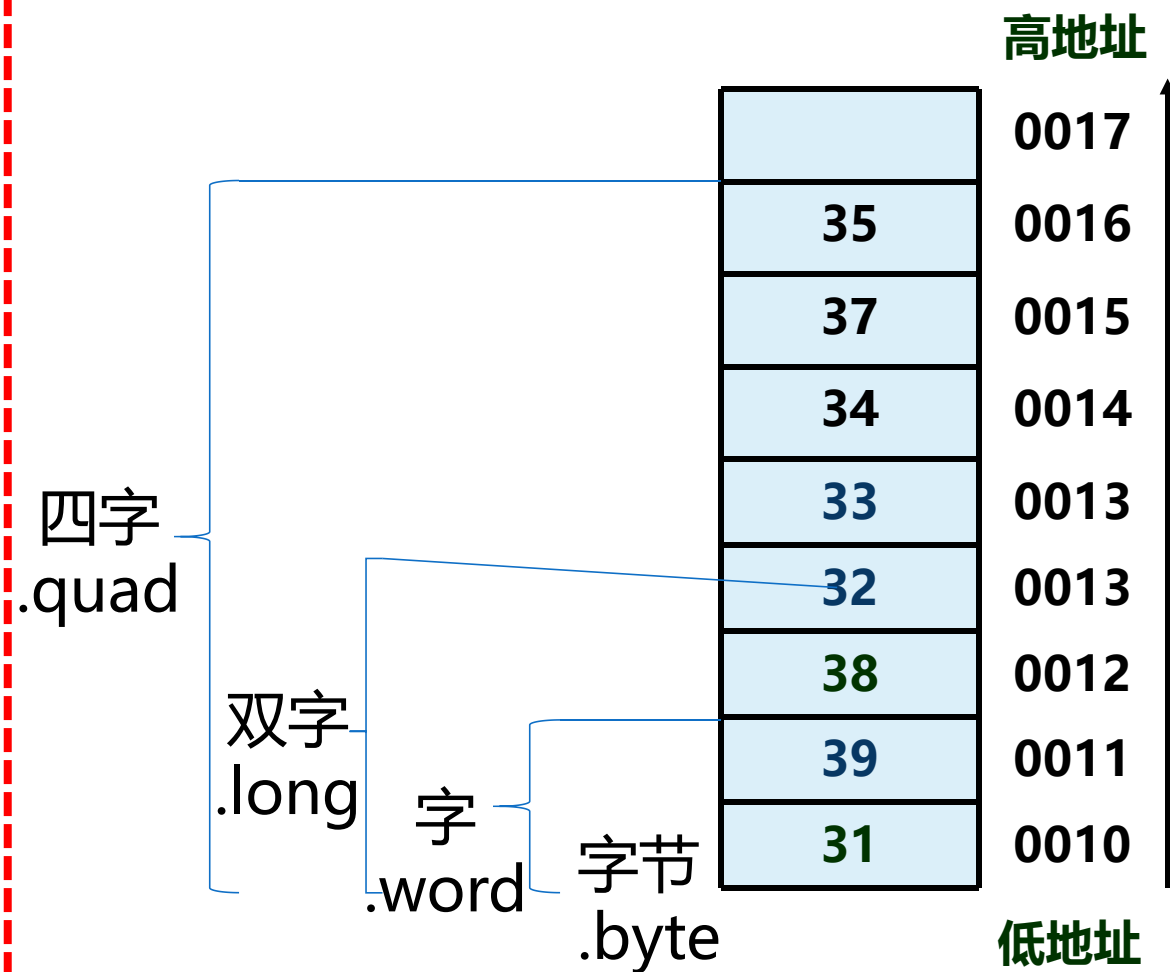


| 变量定义伪指令 | 变量类型 | 功能                                                             |
|---------|------|----------------------------------------------------------------|
| .byte   | 字节   | 分配一个或多个字节单元；数据可以是字节量，也可以是字符串常量。<br>字节量表示8位无符号数或有符号数、字符的ASCII码值 |
| .word   | 字    | 分配一个或多个字单元；数据是字量、16位数据。<br>字量表示16位无符号数或有符号数、16位段选择器、16位偏移地址    |
| .long   | 双字   | 分配一个或多个双字单元；数据是双字量、32位数据<br>双字量表示32位无符号数或有符号数、32位段基地址、32位偏移地址  |
| .quad   | 四字   | 分配一个或多个8字节单元；8字节量表示64位数据                                       |
| .ascii  | 字符串  | 为每个字符串中的每个字符分配一个字节单元                                           |
| .asciz  | 字符串  | 为每个字符串中的每个字符分配一个字节单元，以0结尾                                      |
| .string | 字符串  | 与.asciz功能相同                                                    |

# 大端方式和小端方式



小端方式



大端方式



# 变量定位：对齐地址边界

- N字节数据起始于能够被N整除的地址
  - 2字节、16位数据是被2整除的地址（偶地址）
  - 4字节、32位数据是被4整除的地址（模4地址）
  - 8字节、64位数据是被8整除的地址（模8地址）
- .align 对齐指令
  - 格式： **.align n**     #  $n \in \{2, 4, 8, \dots\}$  且  $n \leq$  当前段对齐属性
  - 功能：将下一数据地址对齐到n的整数倍（地址 % n = 0）

地址A对齐N字节边界

$A \bmod N = 0$  (能够被N整除的地址)

# **.space 填充指令**

- 格式: **.space n [, fill\_value]** # fill\_value默认为0
- 功能: 分配n字节未初始化空间, 可选填充指定值
- 典型用途: 结构体对齐填充、数组/缓冲区预留空间

```
array: .space 1024    # 分配1KB缓冲区  
pad:   .space 3, 0x90 # 填充3字节, 值=0x90
```

# 分析数据表示



在下面括号1-3处填写正确的内容

```
0075 90909090 .data
      90909090
      909090
      .align 8
0000 9CFFFFFF x: .long -100
0004 41         .byte 'A'
              .align 4
( 1 ) 64000000 .long 100
000C ( 2 )     .long -1
0010 000000    ( 3 )
```

答案:

- (1) 0008
- (2) FFFFFFFF
- (3) .space 3

# 第3章 X86-64处理器

- 清楚x86-64处理器的5种工作方式
- 掌握x86处理器支持的数据类型
- 能说出x86-64处理器存储模型名字，并清楚其特点
- 掌握逻辑地址、物理地址的含义及其转换方法
- 掌握x86-64处理器中的寄存器，记住名字和作用
- 掌握各种寻址方式的含义及其在汇编语言中的表达方式
- 掌握存储器寻址方式中有效地址的计算方法

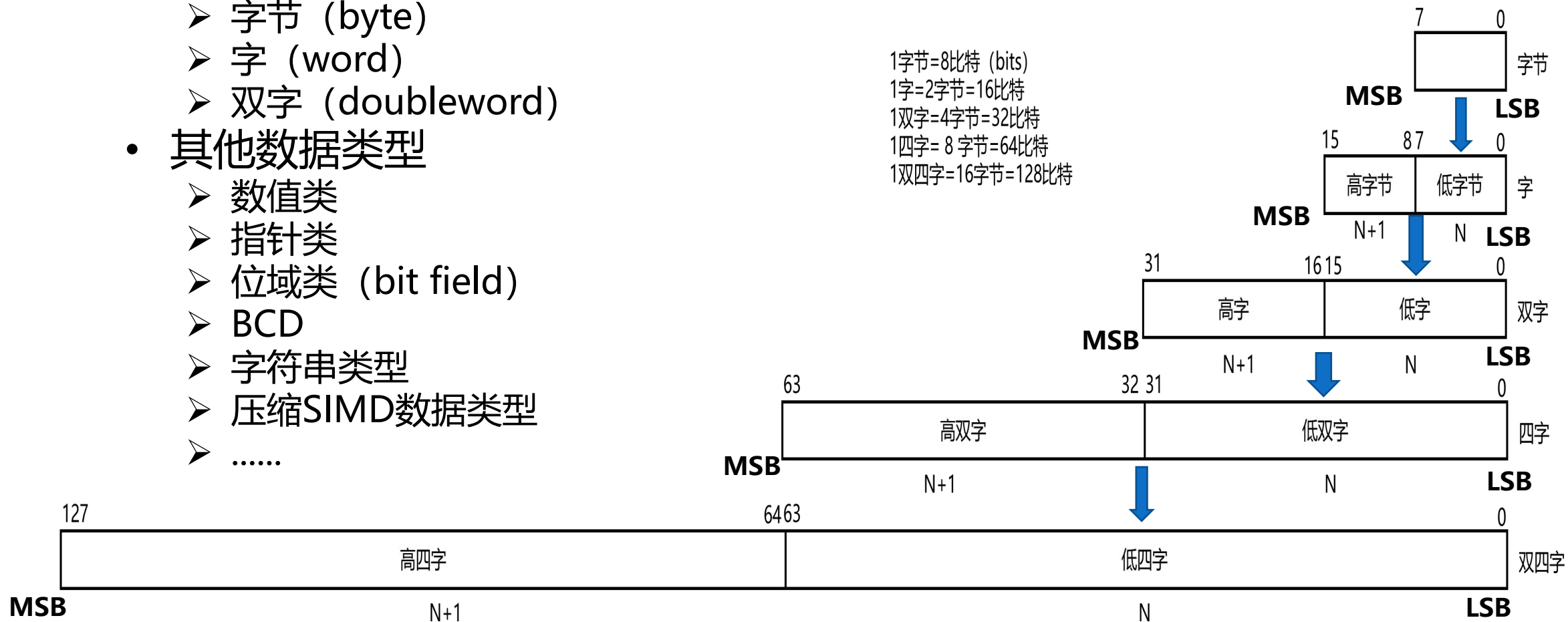
# x86处理器工作方式

- (1) 保护方式 (Protected Mode)
  - IA-32处理器固有的工作状态
  - 具有强大的段页式存储管理和特权与保护能力
  - 使用全部32条地址总线, 可寻址4GB物理存储器
  - 使用平展或段式存储模型
  - 利用虚拟8086方式支持实地址8086软件
- (2) 实地址方式 (Real-address Mode)
  - 可以进行32位处理的快速8086
  - 只能寻址1MB物理存储器空间, 每个段不超过64KB
  - 可以使用32位寄存器、32位操作数和32位寻址方式
  - 只能支持实地址存储模型
- (3) 系统管理方式 (System Management Mode)
  - 提供节能和系统安全管理
- (4) IA-32e模式
  - 兼容模式
  - 64位模式

# x86处理器支持的数据类型

- 基本数据类型
  - 字节 (byte)
  - 字 (word)
  - 双字 (doubleword)
- 其他数据类型
  - 数值类
  - 指针类
  - 位域类 (bit field)
  - BCD
  - 字符串类型
  - 压缩SIMD数据类型
  - .....

1字节=8比特 (bits)  
 1字=2字节=16比特  
 1双字=4字节=32比特  
 1四字= 8 字节=64比特  
 1双四字=16字节=128比特



# x86处理器的存储模型

## (1) 平展存储模型 (Flat Memory Model)

- 存储器是一个连续的地址空间：线性地址空间
- IA-32处理器支持4GB容量线性地址空间

## (2) 段式存储模型 (Segmented Memory Model)

- 存储器由一组独立的地址空间：段 (Segment)
- IA-32处理器每个段都可以达到4GB容量
- 在处理器内部，所有的段都被映射到线性地址空间

## (3) 实地址存储模型 (Real-address Memory Model)

- 8086处理器的存储模型
- 段式存储模型的特例
- 线性地址空间最大为1MB容量，段最大为64KB

# 物理地址与逻辑地址



- 物理地址
  - 物理存储器以字节为基本存储单位，每个存储单元被分配一个唯一的地址
- 程序中使用逻辑地址
  - 在处理器内部、程序员编程时采用的地址
  - 逻辑地址 = 段基地址 : 偏移地址
    - 段基地址 = 在主存中的起始地址
    - 偏移地址 = 距离段基地址的位移量
- 某个存储单元可以有多个逻辑地址，但只有一个唯一的物理地址





# 通用寄存器之间的关系

| D <sub>63</sub> | D <sub>32</sub> D <sub>31</sub> | D <sub>16</sub> D <sub>15</sub> | D <sub>8</sub> D <sub>7</sub> | D <sub>0</sub> | D <sub>0</sub> -D <sub>15</sub> | D <sub>0</sub> -D <sub>31</sub> |
|-----------------|---------------------------------|---------------------------------|-------------------------------|----------------|---------------------------------|---------------------------------|
|                 | RAX                             | AH                              | AL                            |                | AX                              | EAX                             |
|                 | RBX                             | BH                              | BL                            |                | BX                              | EBX                             |
|                 | RCX                             | CH                              | CL                            |                | CX                              | ECX                             |
|                 | RDX                             | DH                              | DL                            |                | DX                              | EDX                             |
|                 | RSI                             |                                 | SIL                           |                | SI                              | ESI                             |
|                 | RDI                             |                                 | DIL                           |                | DI                              | EDI                             |
|                 | RSP                             |                                 | SPL                           |                | SP                              | ESP                             |
|                 | RBP                             |                                 | BPL                           |                | BP                              | EBP                             |
|                 | R8                              |                                 | R8B                           |                | R8W                             | R8D                             |
|                 | R9                              |                                 | R9B                           |                | R9W                             | R9D                             |
|                 | R10                             |                                 | R10B                          |                | R10W                            | R10D                            |
|                 | R11                             |                                 | R11B                          |                | R11W                            | R11D                            |
|                 | R12                             |                                 | R12B                          |                | R12W                            | R12D                            |
|                 | R13                             |                                 | R13B                          |                | R13W                            | R13D                            |
|                 | R14                             |                                 | R14B                          |                | R14W                            | R14D                            |
|                 | R15                             |                                 | R15B                          |                | R15W                            | R15D                            |

## x86-64处理器:

- 16个64位通用寄存器
- 16个32位通用寄存器
- 16个16位通用寄存器
- 20个8位通用寄存器

# 64位模式下字节寄存器访问限制

- 一条指令不能同时引用传统的高字节寄存器和新的字节寄存器
- 一条指令可以同时引用传统的低字节寄存器和新的字节寄存器
  - 传统的高字节寄存器
    - AH、BH、CH、DH
  - 传统的低字节寄存器
    - AL、BL、CL或DL
  - 新的字节寄存器
    - R8B~R15B, DIL, SIL, BPL, SPL

例如: `mov ah, al`  
`mov ah,R8B`  
`mov al,dil`



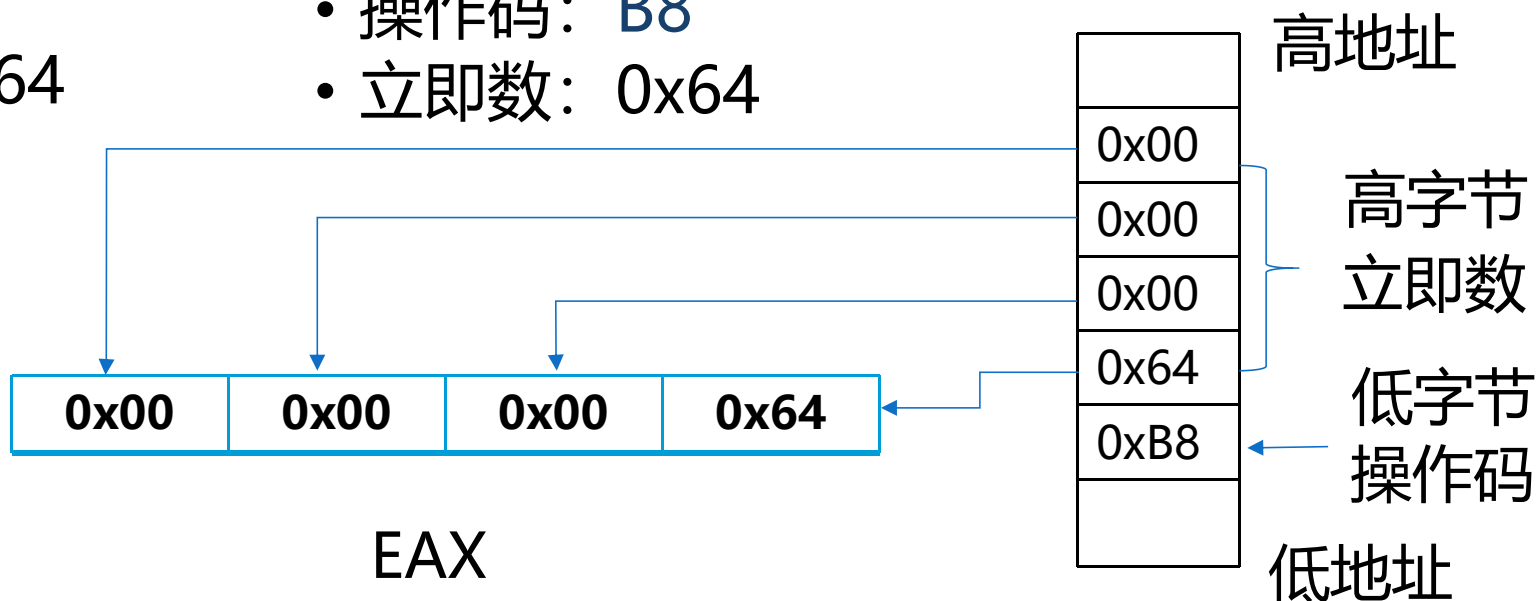
# 处理器专用寄存器



|         |                                    |
|---------|------------------------------------|
| 标志寄存器   | <b>RFLAGS</b>                      |
| 指令指针寄存器 | <b>RIP</b>                         |
| 段寄存器    | <b>CS DS SS</b><br><b>ES FS GS</b> |

# 立即数寻址

- 操作数紧跟操作码，是机器代码的一部分
- 操作数从指令代码中立即得到，即立即数（Immediate），用常量形式直接表达
- 立即数寻址方式只用于源操作数，常用来给寄存器和存储单元赋值
- 例如：**mov eax, 100**
  - 机器代码：B8 64 00 00 00
  - 操作码：B8
  - 立即数：0x00000064
- **mov al, 100**
  - 机器代码：B8 64
  - 操作码：B8
  - 立即数：0x64



# 寄存器寻址

- 操作数存放在处理器的内部寄存器中
- 用寄存器名表示它的内容
- 绝大多数指令采用通用寄存器寻址
- 部分指令支持专用寄存器，例如段寄存器
- 寄存器寻址方式简单快捷，最常使用
- 例如：MOV EBX,EAX

**64位通用寄存器：RAX RBX RCX RDX .....**

**32位通用寄存器：EAX EBX ECX EDX .....**

**16位通用寄存器：AX BX CX DX .....**

**8位通用寄存器：AH AL BH BL .....**

# 偏移地址的组成

64位有效地址 = 基址寄存器 + (变址寄存器 × 比例) + 位移量

- 基址寄存器：任何一个64位通用寄存器之一，RIP
- 变址寄存器：除RSP外的任何64位通用寄存器之一
- 比例：1, 2, 4, 8或16
- 位移量：8, 16, 32或64位有符号值



变化出多种主存寻址方式

# 1. 直接寻址

- 有效地址只有位移量部分，直接包含在指令代码中
- 常用于存取变量
- 仅适用于IA-32处理器保护模式
- 64位模式下不使用这种寻址方式
- 例如：在IA-32处理保护模式下

**MOV ECX,[COUNT]**     #COUNT是变量

**MOV ECX, DS:[405000H]**

- 指令代码：8B 0D 00 50 40 00
- 操作码和寻址方式：8B 0D
- 操作数：有效地址 00405000H



## 2.RIP相对寻址方式

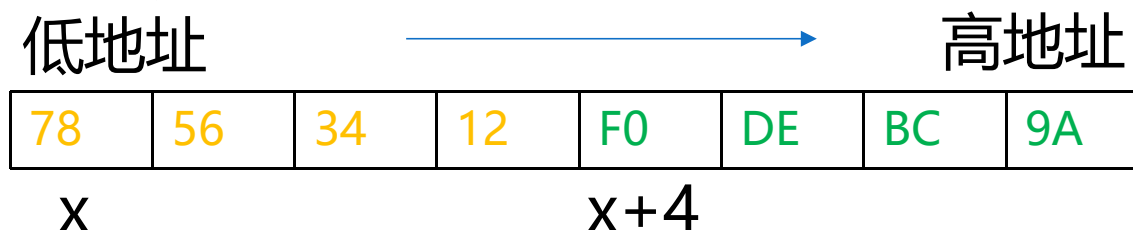
- 64位模式下最常使用的一种存储器寻址方式
- 有效地址 = RIP+64位位移量

行数    线性地址    机器码                      汇编语句

|    |      |          |                                                         |
|----|------|----------|---------------------------------------------------------|
|    |      |          | .data                                                   |
| 9  | 0000 | 78563412 | x: .long 0x12345678,0x9ABCDEF0                          |
| 9  |      | F0DEBC9A |                                                         |
| 10 | 0008 | 31323334 | y: .ascii "12345"                                       |
| 10 |      | 35       |                                                         |
|    |      |          | .text                                                   |
| 14 | 0000 | 8B050000 | mov eax,x[rip] /*有效地址=x+rip,eax=0x12345678 */           |
| 14 |      | 0000     |                                                         |
| 17 | 0010 | 8B050800 | mov eax,y[rip] /*有效地址=y+rip,eax=0x34333231 */           |
| 17 |      | 0000     |                                                         |
| 20 | 0020 | 488B0504 | mov rax,x[rip+4] /*有效地址=x+rip+4,rax=343332319ABCDEF0 */ |
| 20 |      | 000000   |                                                         |

### 3. 寄存器间接寻址

- 有效地址存放在寄存器中(寄存器内容 = 偏移地址 = 有效地址)
- 用中括号括起寄存器
- 可以方便地对数组的元素或字符串的字符进行操作
- 寄存器间接寻址没有说明存储单元类型
- 例如:



```
.data
x: .long 0x12345678,0x9ABCDEF0
.text
lea rax, x[rip] /*x地址复制到rax*/
mov bl, [rax] /*有效地址=rax, bl=0x78 */
mov bx, [rax] /*有效地址=rax, bx=0x5678 */
mov ebx,[rax] /*有效地址=rax, ebx=0x12345678 */
mov rbx,[rax] /*有效地址=rax, rax=0x9ABCDEF012345678 */
```

## 4. 寄存器相对寻址

- 有效地址是寄存器内容与位移量之和
- 可以方便地对数组的元素或字符串的字符进行操作
- 例如：

```
.data  
x: .long 0x12345678,0x9ABCDEF0
```

```
.text
```

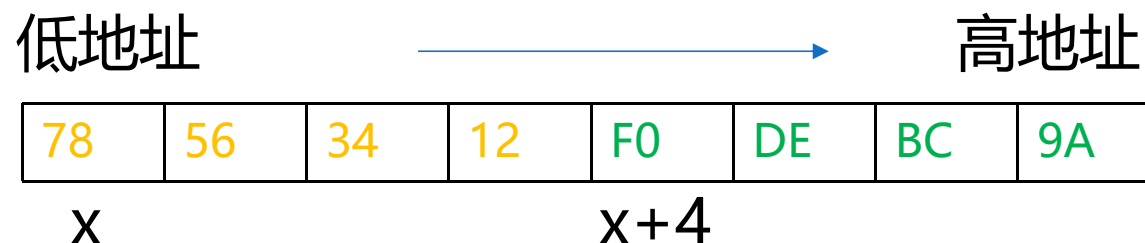
```
lea rax,x[rip] /*x地址复制到rax*/
```

```
mov bl, [rax+1] /*有效地址=rax+1, bl=0x56 */
```

```
mov bx, [rax+2] /*有效地址=rax+2, bx=0x1234 */
```

```
mov ebx,[rax+4] /*有效地址=rax+3, ebx =0x9ABCDEF0 */
```

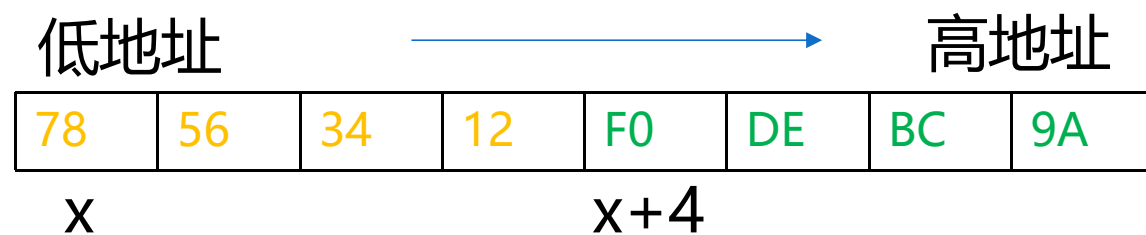
```
mov rbx,[rax+4] /*有效地址=rax+4, rbx =0x000000009ABCDEF0*/
```



**主存以字节为可寻址单位**  
**地址的加减是以字节为单位**

## 5. 基址变址寻址

- 使用变址寄存器寻址操作数
- 便于支持二维数组等数据结构
- 例如：

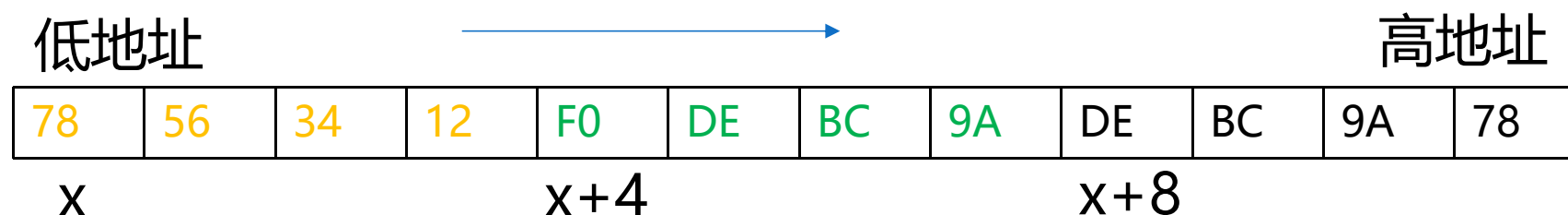


```
.data
x: .long 0x12345678, 0x9ABCDEF0
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 0
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x12345678 */
mov rbx, 4
mov ebx,[rax+rbx] /*有效地址=rax+rbx, ebx=0x9ABCDEF0 */
```

## 6.相对基址变址寻址

- 有效地址由基址寄存器的内容加上变址寄存器内容再加上一个64位的位移量构成

• **有效地址 = 基址寄存器 + 变址寄存器 + 64位位移量**



.data

x: .long 0x12345678, 0x9ABCDEF0, 0x789ABCDE

.text

lea rax, x[rip] /\*x地址复制到rax\*/

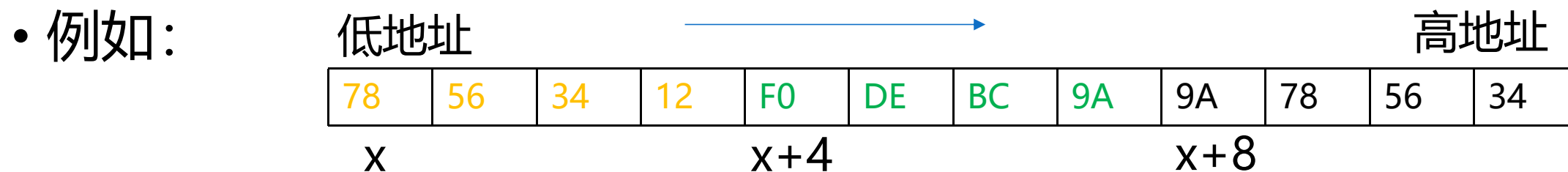
mov rbx, 4

mov eax, [rax+rbx+2] /\*有效地址=rax+rbx+2, eax = 0xBCDE9ABC \*/

mov eax, [rax+rbx+4] /\*有效地址=rax+rbx+4, eax = 0x789ABCDE \*/

## 7. 带比例的基址变址寻址

- 变址寄存器内容乘以比例1（可省略），2，4或8的变址寻址
- 比例1、2、4和8对应8、16、32和64位数据的字节个数，便于以数组元素为单位寻址相应数据



```
.data
x: .long 0x12345678, 0x9ABCDEF0, 0x3456789A
.text
lea rax,x[rip] /*x地址复制到rax*/
mov rbx, 1
mov eax,[rax+rbx*4] /*有效地址=rax+rbx+2, eax = 0x9ABCDEF0 */
mov eax,[ rax+rbx*4+4] /*有效地址=rax+rbx+4, eax = 0x3456789A */
```

# 讨论：源操作数的寻址方式是什么？



//addressing.c

```
....
int main(){
    int i;
    char *p;
    f=1.0;
    for(i=0;i<5;i++)
        array[i] += 10;
    for(p=str;*p;p++)
        *p += 1;
    return 0;
}
```

**mov**    **DWORD PTR f[rip], 0x3f800000**

.....  
**mov**    **eax, 0**

.....  
**lea**    **rcx, array[rip]**  
**movsx** **r8, eax**  
**mov**    **edx, DWORD PTR [rcx+r8\*4]**  
**add**    **edx, 10**

.....  
**lea**    **rdx, str[rip]**  
**add**    **eax, 1**  
**mov**    **BYTE PTR [rdx], al**  
**add**    **rdx, 1**  
**movzx** **eax, BYTE PTR [rdx]**

....

立即数寻址

立即数寻址

RIP相对寻址

寄存器寻址

带比例基址变址寻址

立即数寻址

RIP相对寻址

立即数寻址

寄存器寻址

立即数寻址

寄存器间接寻址

gcc -S -Og -masm=intel -c -o addressing.S addressing.c

# 寻址的数据是什么？



**dvar: .long 0x12345678,12**

lea rbx, dvar[rip]

RBX = 0x00405000

mov eax, [rbx]

EAX = 0x12345678

mov ecx, dvar[rip]

ECX = 0x12345678

mov rsi, 2

RSI = 2

mov edi, [rbx+rsi\*2]

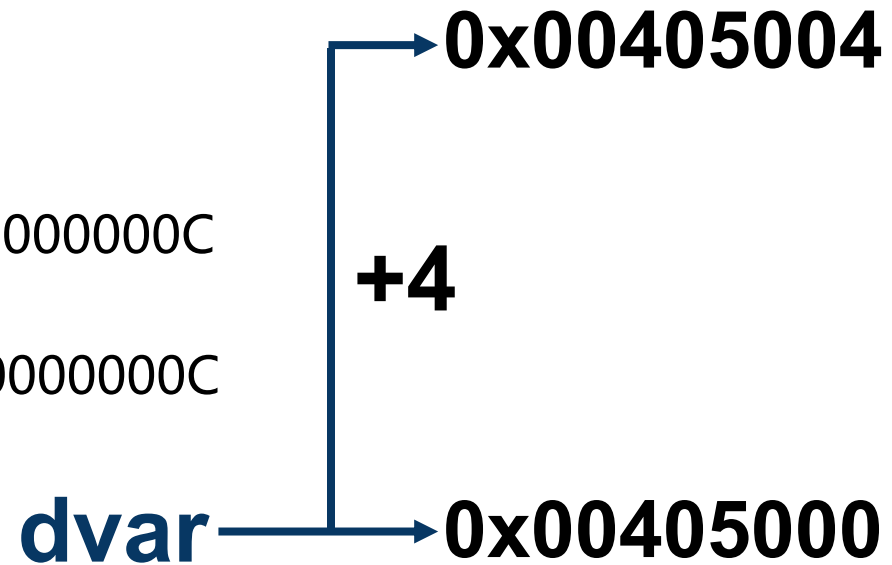
EDI = 0x0000000C

mov edx, [rbx+rsi+2]

EDX = 0x0000000C

mov al, byte ptr dvar[rip+2]

AL = 0x34



地址

|    |
|----|
| 00 |
| 00 |
| 00 |
| 0C |
| 12 |
| 34 |
| 56 |
| 78 |
|    |



## 第4章 X86通用数据处理指令

- 能正确书写数据传送、算术运算类指令、位操作类指令
- 会判断指令执行结果对标志位的影响
- 能阅读分析数据处理类的应用程序
- 能编写指令序列完成数据处理类的应用
- 能写出C语言程序对应的汇编指令
- 能写出汇编指令序列对应的C语句

# 注意1：双操作数必须类型一致

|                            |                                              |
|----------------------------|----------------------------------------------|
| <code>mov esi,dl</code>    | <p>#错误：类型不一致</p> <p>#esi为32位寄存器，dl为8位寄存器</p> |
| <code>mov esi,edx</code>   | <p>#正确：两个32位寄存器传送</p>                        |
| <code>mov al,0x050a</code> | <p>#错误：类型不一致</p> <p>#0x050a超过了寄存器AL范围</p>    |
| <code>mov eax,#050a</code> | <p>#正确：双字量数据传送</p>                           |

## 注意2：操作数必须有明确的类型

`mov [rbx], 255` #错误：无明确类型

`mov byte ptr [rbx], 255` #正确：BYTE PTR说明是字节操作

`mov word ptr [ebx], 255` #正确：WORD PTR说明是字操作

`mov dword ptr [ebx], 255` #正确：DWORD PTR说明是双字操作

# 注意3：双操作数不允许都是主存单元



#假设dbuf1和dbuf2是两个双字变量

`mov dbuf2[rip],dbuf1[rip]` #错误：两个操作数都是存储单元

`mov eax,dbuf1[rip]` #正确： $eax = dbuf1$ （将dbuf1内容送eax）

`mov dbuf2[rip],eax` #正确： $dbuf2 = eax$ （将eax内容送dbuf2）

## 注意4：不可随意操作专用寄存器

mov ds,100

#错误：立即数不能直接传送段寄存器

mov ax,cs

mov ds,ax

#正确：通过AX间接传送给DS

# 注意5：新字节寄存器的用法



- 不能同时使用传统的高字节和新的字节寄存器

|                           |                       |
|---------------------------|-----------------------|
| <code>mov dil, al</code>  | #正确, 新字节寄存器和传统低字节寄存器  |
| <code>mov dil, ah</code>  | #错误: 不能同时使用ah和新增字节寄存器 |
| <code>mov bl, ah</code>   | #正确, 两个传统字节寄存器        |
| <code>mov dil, r8b</code> | #正确, 两个新字节寄存器         |

# 课堂练习：下面指令是否正确

- 1 `mov dvar[rip+4],dvar[rip]`
- 2 `mov edi,si`
- 3 `mov eax,0x50a`
- 4 `mov al,0x50a`
- 5 `mov dword ptr [rbx],255`
- 6 `mov [rbx+4],200`
- 7 `mov ds, 100`
- 8 `mov ax, ds`
- 9 `mov edx, [ebx+0xff]`
- 10 `mov [rsi], dword ptr [rdi]`
- 11 `mov ecx, cs`
- 12 `xchg al, byte ptr dvar[rip+3]`
- 13 `xchg 100, al`

- |    |   |
|----|---|
| 1  | × |
| 2  | × |
| 3  | √ |
| 4  | × |
| 5  | √ |
| 6  | × |
| 7  | × |
| 8  | √ |
| 9  | √ |
| 10 | × |
| 11 | × |
| 12 | √ |
| 13 | × |

# 〔例4-1〕：C语言中的赋值语句



```
/* eg0401.c*/  
long exchange(long *xp, long y)  
{  
    long x = *xp;  
    *xp = y;  
    return x;  
}
```

xp: rcx, x: eax, y: edx



```
mov    eax, DWORD PTR [rcx]  
mov    DWORD PTR [rcx], edx
```



# 地址传送指令

- 地址传送指令获取存储器操作数的地址  
LEA r16/r32/r64,mem  
#r16/r32/r64←mem的有效地址EA (不需类型一致)  
#r16:实地址模式, r32: IA-32e , r64: 64位模式
- LEA指令类似的地址操作符OFFSET的作用
  - LEA指令在指令执行时计算出偏移地址
  - OFFSET操作符在汇编阶段取得变量的偏移地址
  - OFFSET无需在执行时计算、指令执行速度更快
  - LEA指令能获取汇编阶段无法确定的偏移地址

**lea rsi, var[rip] #64位模式推荐**

**mov rdi, offset var #仅用于intel语法, 不稳定, 避免使用**

## 【例4-2】：LEA指令实现算术运算

地址传送指令经常用于表达式求值。

```
/*eg0402.c*/  
int scale(int x, int y, int z){  
    long t=x+4*y+12*z;  
    return t;  
}
```

x: rcx  
y: rdx  
z: r8



```
lea    eax, [rcx+rdx*4]    #eax= x+4*y  
lea    edx, [r8+r8*2]      #edx=3*z  
lea    eax, [rax+rdx*4]    #eax=x+4*y+(3*z)*4
```

# 【例4-5】：C语言中的加法运算



```
//eg0405.c
#include <stdio.h>
int x=123456789, y=-978522123;
char c='A';
int main(){
printf("x+y =%d\n", x+y+250);
printf("c =%c\n",c+32);
return 0;
}
```



```
#x+y+250
mov     edx, DWORD PTR y[rip]
add     edx, DWORD PTR x[rip]
add     edx, 250
#c+32
movsx   edx, BYTE PTR c[rip]
add     edx, 32
```

# 〔例4-17〕 C语言的算术运算



```
long arith(long x, long y, long z)
{
    long t1 = x ^ y;
    long t2 = z * 48;
    long t3 = t1 & 0x0F0F0F0F;
    long t4 = t2 - t3;
    return t4;
}
```

ecx:x, edx:y, r8:z

|     |                |                         |
|-----|----------------|-------------------------|
| xor | ecx, edx       | # t1 = x ^ y            |
| lea | eax, [r8+r8*2] | # 3×z                   |
| sal | eax, 4         | # t2 = z ×48            |
| and | ecx, 252645135 | # t3 = t1 & 0x0F0F0F0F; |
| sub | eax, ecx       | # t4 = t2 - t3;         |

# 移位指令



- 逻辑左移指令 SHL
- 逻辑右移指令 SHR
- 算术左移指令 SAL
- 算术右移指令 SAR
- 双精度左移指令 SHLD
- 双精度右移指令 SHRD
  
- 不带进位循环左移指令 ROL
- 不带进位循环右移指令 ROR
- 带进位循环左移指令 RCL
- 带进位循环右移指令 RCR

**S: Shift**

**R: Rotate**

**L: Left**

**R: Right**

**A: Arithmetic**

**C: Carry flag**

**D: 双精度**

# 用移位指令实现算术运算



- 逻辑左移一位，相当于无符号数乘以2，逻辑左移n位，相当于无符号数乘以 $2^n$ ；
- 逻辑右移一位，相当于无符号数除以2，逻辑右移n位，相当于无符号数除以 $2^n$ ；
- 商在目的操作数中，余数由CF标志反映。
- 算术右移一位，相当于有符号数除以2，算术右移n位，相当于有符号数除以 $2^n$ 。
- 当操作数为负数，并且最低位有1移出时，算术右移与等效的IDIV指令的结果不同。

例如：EDX $\leftarrow$ 3 $\times$ EAX + 7 $\times$ EBX

分析：3 $\times$ EAX + 7 $\times$ EBX = 2 $\times$ EAX + 1 $\times$ EAX + 8 $\times$ EBX - 1 $\times$ EBX

```
mov esi,eax
shl esi,1    #esi $\leftarrow$ 2 $\times$ eax
add esi,eax  #esi $\leftarrow$ 3 $\times$ eax
mov edx,ebx
shl edx,3    #edx $\leftarrow$ 8 $\times$ ebx
sub edx,ebx  #edx $\leftarrow$ 7 $\times$ ebx
add edx,esi  #edx $\leftarrow$ 7 $\times$ ebx + 3 $\times$ eax
```

# 第5章 X86汇编语言程序结构

- 能正确书写转移指令，掌握转移指令的功能和格式
- 会正确判断转移发生的条件，会阅读分析分支结构程序的功能、说明运行结果
- 会正确使用转移指令实现单分支和双分支结构
- 掌握循环指令loop的功能
- 会阅读分析循环结构程序的功能、说明运行结果，会正确使用转移指令和循环指令实现循环结构
- 掌握call 和ret指令的功能
- 能画出堆栈在子程序调用和返回中的变化，清楚子程序调用过程中RIP寄存器的变化
- 掌握三种参数传递方式
- 会阅读分析子程序结构程序的功能、说明运行结果，会编写子程序结构的程序
- 掌握模块间共享变量或子程序，文件包含，宏定义的伪指令
- 掌握模块连接的过程和注意事项，清楚子程序库生成和调用的过程及其注意事项
- 会分析多模块程序的功能和执行结果，并能说明开发过程
- 掌握宏定义和宏调用的方法，清楚宏展开的过程和结果，会编写宏并调用
- 清楚宏与子程序的区别

# 两条基本的转移指令

- 程序顺序执行，RIP自动指向下一条指令
- 转移指令改变程序执行顺序，即改变RIP[+CS]

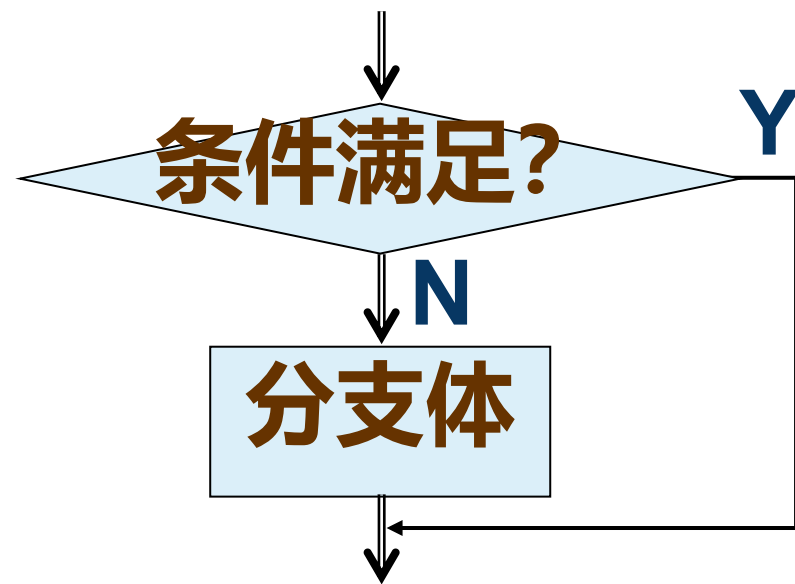
**JMP label** #无条件转移指令

- 类似C/C++的goto语句

**Jcc label** #有条件转移指令

- 类似C/C++的if语句

**Jcc: jump with condition**







# 转移条件cc: 单个标志状态

|         |      |                               |
|---------|------|-------------------------------|
| JZ/JE   | ZF=1 | Jump if Zero/Equal            |
| JNZ/JNE | ZF=0 | Jump if Not Zero/Not Equal    |
| JS      | SF=1 | Jump if Sign                  |
| JNS     | SF=0 | Jump if Not Sign              |
| JP/JPE  | PF=1 | Jump if Parity/Parity Even    |
| JNP/JPO | PF=0 | Jump if Not Parity/Parity Odd |
| JO      | OF=1 | Jump if Overflow              |
| JNO     | OF=0 | Jump if Not Overflow          |
| JC      | CF=1 | Jump if Carry                 |
| JNC     | CF=0 | Jump if Not Carry             |



# 转移条件cc: 两数大小关系

|         |            |                                   |
|---------|------------|-----------------------------------|
| JB/JNAE | CF=1       | Jump if Below/Not Above or Equal  |
| JNB/JAE | CF=0       | Jump if Not Below/Above or Equal  |
| JBE/JNA | CF=1或ZF=1  | Jump if Below/Not Above           |
| JNBE/JA | CF=0且ZF=0  | Jump if Not Below or Equal/Above  |
| JL/JNGE | SF≠OF      | Jump if Less/Not Greater or Equal |
| JNL/JGE | SF=OF      | Jump if Not Less/Greater or Equal |
| JLE/JNG | ZF≠OF或ZF=1 | Jump if Less or Equal/Not Greater |
| JNLE/JG | SF=OF且ZF=0 | Jump if Not Less or Equal/Greater |

# 转移条件cc: RCX、ECX是否为0

- JECXZ指令对ECX寄存器进行测试, 当ECX=0时, 转移到目标地址处。
- JRCXZ指令对RCX寄存器进行测试, 当RCX=0时, 转移到目标地址处。

JECXZ/JRCXZ label

#ECX/RCX = 0, 转移; 否则顺序执行

```
CMP ECX/RCX, 0  
JZ label
```

# 1 循环指令



## LOOP label

#RCX/ECX/CX ← RCX/ECX/CX - 1; 若RCX/ECX/CX ≠ 0, 循环到LABEL  
#否则, 顺序执行

## LOOPZ/LOOPE label

# RCX/ECX/CX ← RCX/ECX/CX - 1 ;  
# RCX/ECX/CX ≠ 0 且 ZF = 1, 循环到label  
#否则, 顺序执行

## LOOPNZ/LOOPNE label

# RCX/ECX/CX ← RCX/ECX/CX - 1 ;  
#若RCX/ECX/CX ≠ 0 且 ZF = 0, 循环到label  
#否则, 顺序执行

- 目标地址采用相对短转移
- 实模式: CX, 32位保护模式: ECX, 64位模式: RCX

DEC RCX/ECX/CX  
JNZ label

JNZ not\_label  
DEC RCX/ECX/CX  
JNZ label

JZ not\_label  
DEC RCX/ECX/CX  
JNZ label

.text

**子程序名:** #子程序开始  
#子程序体  
**ret** #子程序返回

main: #程序入口  
#主程序结束

**子程序名:** #子程序开始  
#子程序体  
**ret** #子程序返回

# 1. 子程序调用指令CALL

- CALL指令用在主程序中，实现子程序的调用
  - 分成段内调用（近调用）和段间调用（远调用）
  - 64位模式下，目标地址采用相对寻址或间接寻址
  - 32位兼容模式下，目标地址可以采用相对寻址、间接寻址、或直接寻址
- CALL指令的两个功能：
  - 入栈返回地址：将CALL下条指令的地址压入堆栈
  - 目标地址送RIP  $RSP \leftarrow RSP - 8, SS:[RSP] \leftarrow RIP, RIP \leftarrow \text{目标地址}$ 
    - 实模式：16位，32位保护模式：32位

## 2. 子程序返回指令RET

- RET指令用在子程序结束，实现返回主程序

RET

#无参数返回：出栈返回地址

RET i16

#有参数返回：① 出栈返回地址

②  $RSP \leftarrow RSP + i16$

汇编程序会根据存储模型等信息确定子程序的远近调用，并相应产生返回指令

# 寄存器传递参数

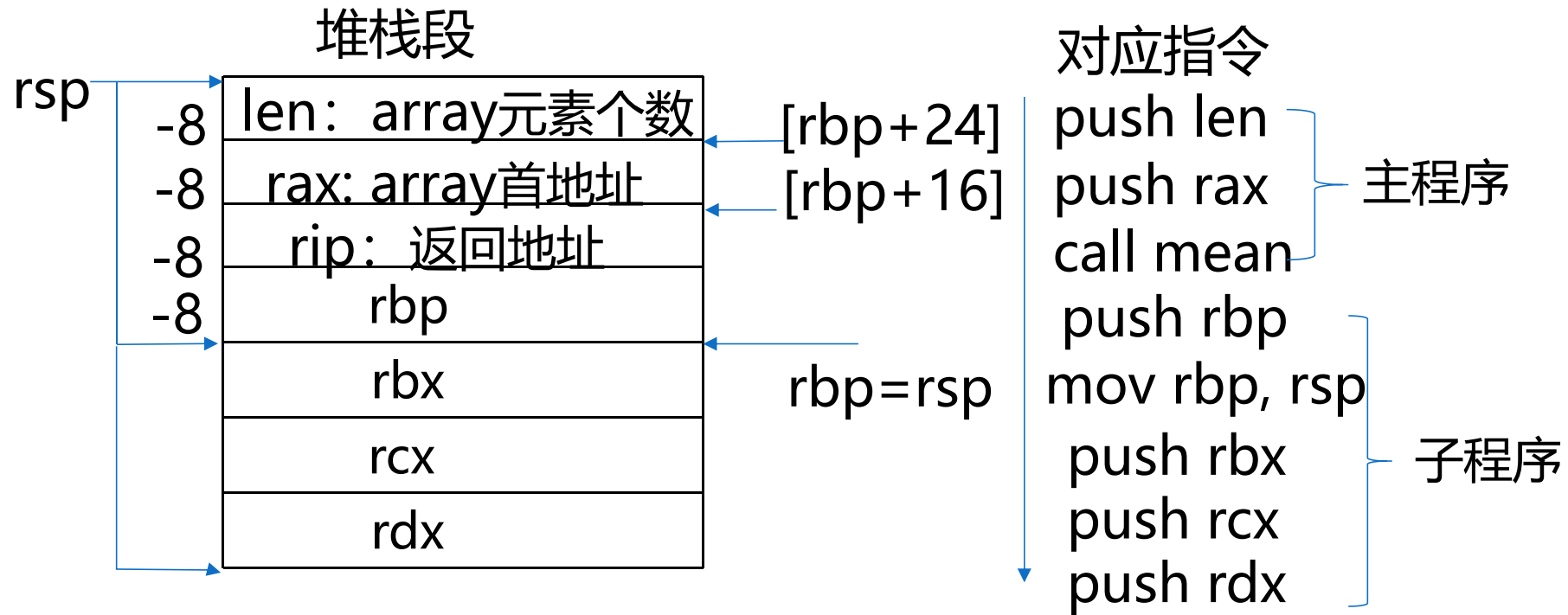
- 最简单和常用的参数传递方法
- 把参数存于约定的寄存器
  - 少量数据直接传递数值
  - 大量数据只能传递地址
- 带有出口参数的寄存器不能保护和恢复
- 带有入口参数的寄存器可以保护、也可以不保护，但最好能够保持一致



# 共享变量传递参数

- 子程序和主程序使用同一个变量名存取数据
- 如果变量定义和使用不在同一个程序模块中
  - 需要利用`.global`、`.extern`声明
- 共享变量传递参数，子程序的通用性较差
- 特别适合在多个程序段间、尤其在不同的程序模块间传递数据

# 【例5-25】堆栈传递参数



# 【例5-25】平衡堆栈

等价于子程序中 ret 16

主程序 { 对应指令  
add rsp,16

子程序 {  
ret  
pop rdx  
pop rcx  
pop rbx  
pop rbp

| 字节数   | 堆栈段            |
|-------|----------------|
| +16 8 | len: array元素个数 |
| 8     | rax: array首地址  |
| +8 8  | rip: 返回地址      |
| 8     | rbp            |
| +32 8 | rbx            |
| 8     | rcx            |
| 8     | rdx            |
| rsp 8 |                |

# 模块连接的操作过程

- 子程序单独编写一个源程序文件
- 子程序源文件汇编形成目标模块OBJ文件
- 连接时输入子程序目标模块文件名

## 操作过程

```
gcc -c -o eg0528.o ./progs/eg0528.s
```

```
gcc -c -o eg0528s.o ./progs/eg0528s.s
```

```
gcc -o eg0528 eg0528.o eg0528s.o
```

或:

```
>gcc -o eg0528 ./progs/eg0528.S ./progs/eg0528s.S
```

```
>eg0528
```

```
RFLAGS =00000A12 O
```

# 子程序库的使用



- 子程序单独编写一个源程序文件
- 子程序源文件汇编形成目标模块文件
- 利用库管理工具把子程序模块加入到子程序库
- 在连接主程序时提供子程序库文件名

操作过程

```
ar rcs eg0528.a ./progs/eg0528s.o
```

```
gcc -o eg0528 ./progs/eg0528.s eg0528.a
```

- 大型源程序可以合理地分放在若干个文本文件中
  - 各种常量定义、声明语句等组织在包含文件 (\*.inc)
  - 常用的或有价值的宏定义存放在宏定义文件 (\*.mac)
  - 常用的子程序形成汇编语言源文件 (\*.asm, \*.s)
  - 任何文本文件

➤使用源文件包含伪指令 `.include`

**.include** 文件名

#将指定文件内容插入主体源程序文件

# 源文件包含的使用



- 被包含文件
  - 文件名要符合操作系统规范
  - 只能是文本文件
  - 内容被插入源文件包含.include语句所在的位置
- 实质仍然是一个源程序
  - 只是分开在若干个文件中
  - 只需针对主体源程序文件进行汇编、连接

操作过程

```
gcc -o eg0528_2 eg0528_2.s
```

# 1. 宏定义和宏调用

- 宏定义

```
.macro WriteString msg  
    push rax  
    lea rax, \msg[rip]  
    call dispmsg  
    pop rax  
.endm
```

- 宏调用

```
WriteString msg
```

- 宏展开

```
push rax  
lea rax, msg[rip]  
call dispmsg  
pop rax
```



## 【例5-29】 状态标志显示宏

- 利用处理器指令PUSHF可以将当前标志寄存器内容压入堆栈，然后从堆栈弹出就能够逐个显示状态标志OF、SF、ZF、AF、PF和CF，每个标志依次对应标志寄存器的D11、D7、D6、D4、D2和D0位。将标志位移入CF、加0x30转换为ASCII码。

```
.macro rfbt bit1, bit2
    xor ebx,ebx                # EBX清0，用于保存字符
    rol eax,\bit1             # 将某个标志左移bit1位，进入最低位
    adc ebx,0x30               # 转换为ASCII字符
    mov rfmmsg[rip+\bit2],bl  # 保存于字符串bit2位置
.endm
```

## 【例5-29】



.data

rfmsg: .asciz "OF=0, SF=0, ZF=0, AF=0, PF=0, CF=0 \n" #初始化

.text

.globl main

main: # 程序执行起始位置

sub rsp, 40

mov al, 50

sub al, 80 #假设一个运算

pushfq #将标志位寄存器的内容压入堆栈

pop rax #将标志位寄存器的内容存入RAX

# 〔例5-29〕 宏调用



**rfbit 21,3**      #显示OF (原来的OF需左移21位, 进入当前CF)  
**rfbit 4,9**      #显示SF (原来的SF再左移4位, 进入当前CF)  
**rfbit 1,15**     #显示ZF (原来的ZF再左移1位, 进入当前CF)  
**rfbit 2,21**     #显示AF (原来的AF再左移2位, 进入当前CF)  
**rfbit 2,27**     #显示PF (原来的PF再左移2位, 进入当前CF)  
**rfbit 2,33**     #显示CF (原来的CF再左移2位, 进入当前CF)

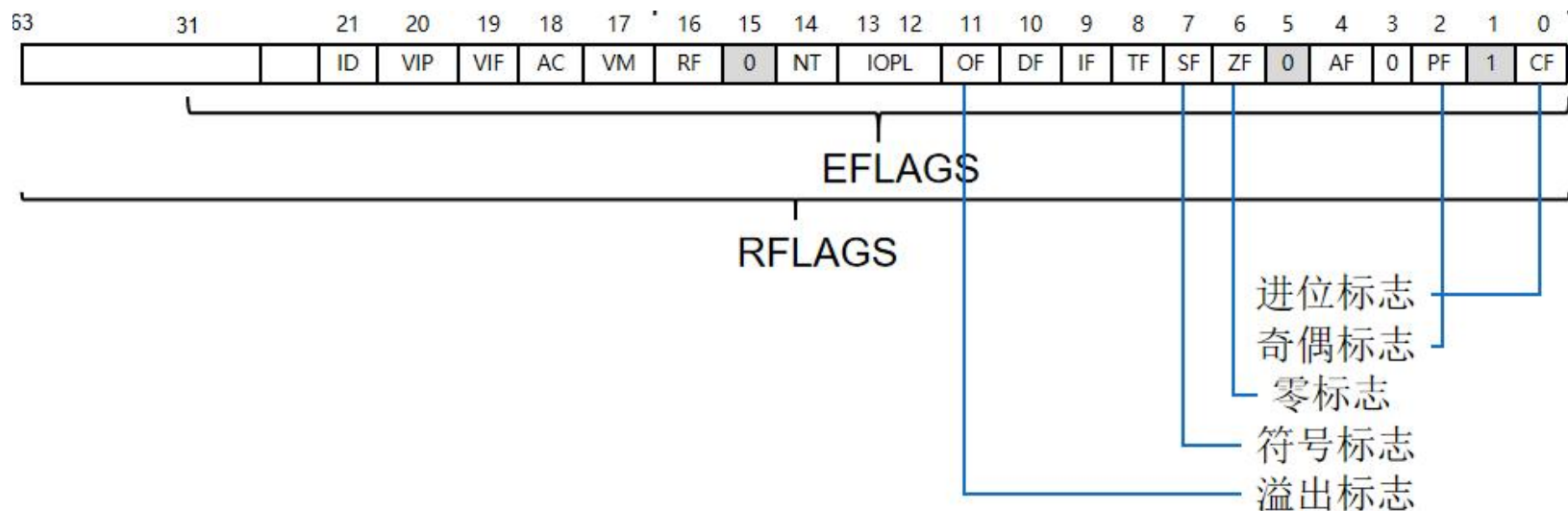
lea rax,rfmsg[rip]

call dispmsg

mov eax,0

**add rsp, 40**

ret



# 宏与子程序：简化程序



- 宏仅是源程序级的简化

- 宏调用在汇编时进行程序语句的展开，不需要返回
- 不减小目标程序，执行速度没有改变

- 子程序不仅简化源程序，还是目标程序级的简化

- 子程序调用在执行时由CALL指令转向、RET指令返回
- 形成的目标代码较短，执行速度减慢

# 宏与子程序：传递参数



- 宏通过形参、实参结合实现参数传递
  - 使用软件方法，简洁直观、灵活多变
  - 传递出错多体现为语法错误，易于发现
- 子程序利用寄存器、存储单元或堆栈等实现
  - 运用硬件本身，规则严格、方法固定
  - 传递出错常反映为逻辑或运行错误，较难排除



# 宏与子程序：选用原则



## • 选用宏

- 当程序段较短或要求较快执行时
- 宏常依附于源程序，适合进行全局性预处理



## • 选用子程序

- 当程序段较长或为减小目标代码时
- 子程序更具有独立性，可以分别编写

# 阅读程序



阅读源程序，其中子程序counta统计某个字符串中大写字母“A”的数量，主程序显示统计的结果。首先，填空（1）～（5）将程序补充完整。然后，画出从主程序压入参数，到进入子程序、压入寄存器保护之后的堆栈区域情况，要求以8字节为单位给出堆栈保存的数据（含义），并表示出rbp指向的位置。

```
;数据段
msg .asciz 'Try to make it.'
.equ count, (. - msg)
;代码段，主程序
push count
lea rax, msg[rip]
push rax
call counta
add rsp, (1)
call dispuid
;以十进制形式显示EAX内数值
```

;代码段，子程序  
counta:

```
push rbp
mov rbp, rsp
push rbx
push rcx
mov rbx, (2)
mov rcx, (3)
mov rax, 0
again: cmp [rbx], 'A'
jne next
inc rax
next: inc rbx
loop again
pop rcx
(4)
(5)
ret
```

- (1) 16
- (2) [rbp+16]
- (3) [rbp+24]
- (4) pop rbx
- (5) pop rbp





# 写出实现下述功能的汇编语言程序



```
//eg0504b.c
int sub_of(int x, int y, int *diff){
*diff=x-y;
return 0;
}

int main(){
int d,f;
f=sub_of(1234567890,-9999999999,&d);
if(!f)
printf("x-y = %d = %u = %x",d,d,d);
else
printf( "溢出");
}
```

gcc -o eg0504b eg0504b.c

# 第6章 鲲鹏处理器汇编语言



- 能说出鲲鹏处理器的特点及与ARM汇编的关系
- 能对比x86架构和AArch64架构汇编的区别